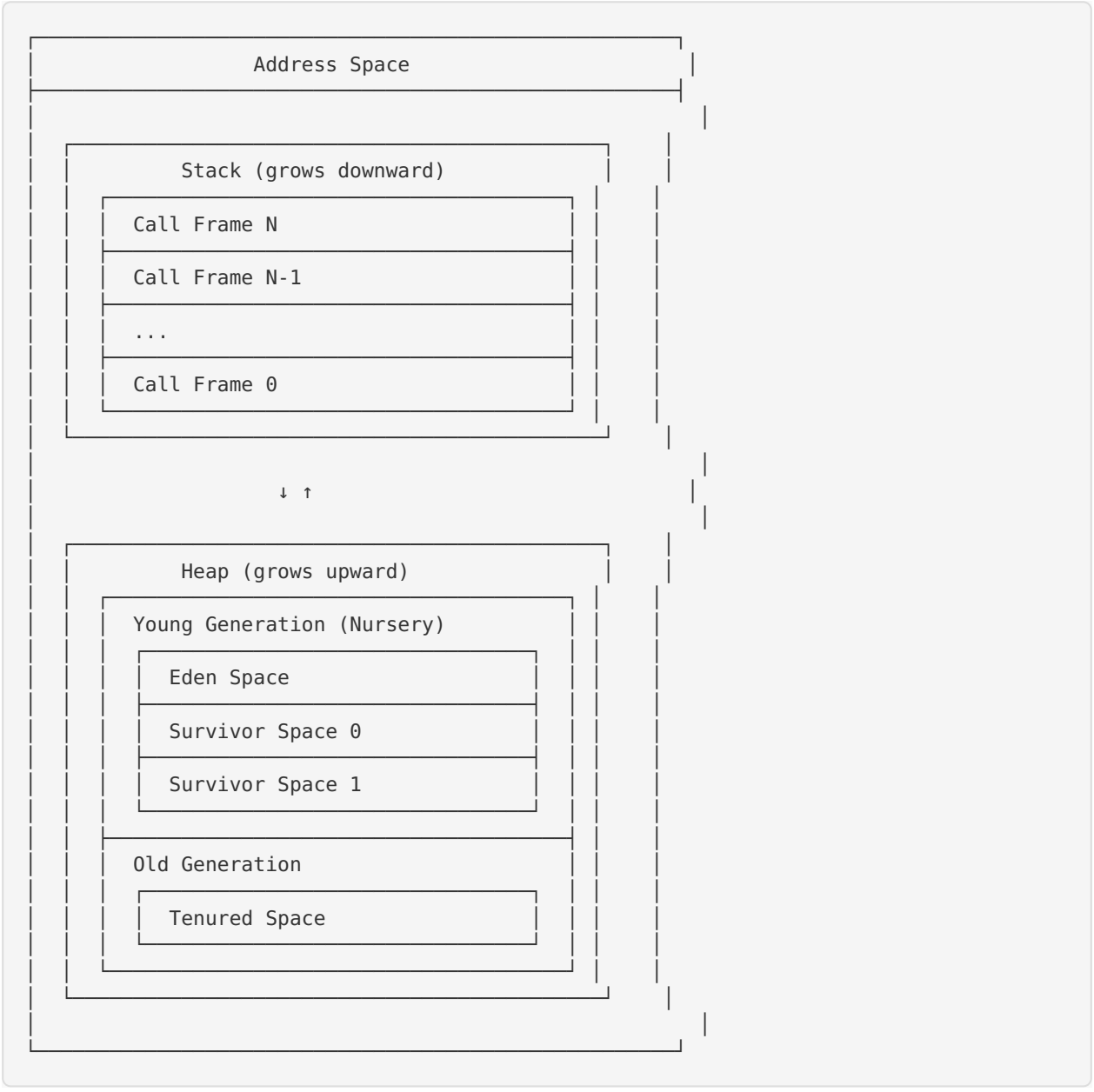
Memory Model and Garbage Collection

Overview

The BDI Kernel uses sophisticated memory management with automatic garbage collection to provide safe, efficient memory allocation and deallocation. The system employs a generational garbage collector that balances low pause times with high throughput.

Memory Architecture

Memory Layout



Memory Regions

Stack:
- Fixed size per thread
- Automatic allocation/deallocation
- Fast access (CPU cache-friendly)
- Stores local variables and call frames

Heap:
- Dynamic size
- Manual allocation, automatic deallocation (GC)
- Stores objects and data structures
- Divided into generations

Generational Garbage Collection

Generational Hypothesis

The generational GC is based on the **weak generational hypothesis**:

> Most objects die young.

Implications:
- Frequent collection of young generation
- Infrequent collection of old generation
- Low pause times for most collections

Generation Structure

Young Generation (Nursery)

Purpose: Fast allocation for short-lived objects.

Structure:

```
typedef struct {
    void* eden_start;
    void* eden_end;
    void* eden_top;

    void* survivor0_start;
    void* survivor0_end;
    void* survivor0_top;

    void* survivor1_start;
    void* survivor1_end;
    void* survivor1_top;

    uint8_t active_survivor;  // 0 or 1
} YoungGeneration;
```

Spaces:
- **Eden**: Initial allocation space
- **Survivor 0**: First survivor space
- **Survivor 1**: Second survivor space

Allocation:
- Bump pointer allocation in Eden
- Very fast (just increment pointer)
- No fragmentation

Collection:
- Copy collection algorithm
- Frequent (every few MB allocated)
- Fast (1-10 ms)

Old Generation (Tenured)

Purpose: Long-lived objects.

Structure:

```
typedef struct {
    void* start;
    void* end;
    void* top;

    FreeList* free_list;
    size_t used_bytes;
    size_t total_bytes;
} OldGeneration;
```

Allocation:
- Free list allocation
- Slower than young generation
- May have fragmentation

Collection:
- Mark-and-sweep algorithm
- Infrequent (when old gen fills)
- Slower (10-100 ms)

Object Structure

```
typedef struct GCObject {
    struct GCObject* next;     // For free list
    uint32_t type_id;          // Object type
    uint32_t size;             // Object size
    uint8_t age;               // Generational age
    uint8_t marked;            // Mark bit for GC
    uint8_t generation;        // 0=young, 1=old
    uint8_t padding;

    // Object data follows
    uint8_t data[];
} GCObject;
```

Header Size: 24 bytes (on 64-bit systems)

Alignment: 8-byte aligned for performance

Garbage Collection Algorithms

Young Generation Collection (Minor GC)

Algorithm: Cheney's copying collector

Process:
1. **Scan Roots**: Identify root objects (stack, globals)
2. **Copy Live Objects**: Copy live objects from Eden and active survivor to inactive survivor
3. **Update References**: Update all references to copied objects
4. **Promote Old Objects**: Move objects that survived N collections to old generation
5. **Swap Survivors**: Make inactive survivor the active one
6. **Reset Eden**: Reset Eden allocation pointer

Pseudocode:

```
void minor_gc(GenerationalGC* gc) {
    // 1. Scan roots
    for (root in gc->roots) {
        if (is_young_generation(root)) {
            copy_object(root, survivor_space);
        }
    }

    // 2. Scan remembered set (old-to-young references)
    for (ref in gc->remembered_set) {
        if (is_young_generation(ref->target)) {
            copy_object(ref->target, survivor_space);
        }
    }

    // 3. Scan survivor space (breadth-first)
    while (scan_ptr < survivor_top) {
        object = *scan_ptr;
        for (field in object->fields) {
            if (is_young_generation(field)) {
                copy_object(field, survivor_space);
            }
        }
        scan_ptr++;
    }

    // 4. Promote old objects
    for (object in survivor_space) {
        if (object->age >= PROMOTION_THRESHOLD) {
            promote_to_old_generation(object);
        }
    }

    // 5. Swap survivors
    swap_survivor_spaces();

    // 6. Reset Eden
    eden_top = eden_start;
}
```

Performance:
- Pause time: 1-10 ms

- Throughput: 95-99% (5-1% GC overhead)
- Frequency: Every 1-10 MB allocated

Old Generation Collection (Major GC)

Algorithm: Mark-and-sweep with optional compaction

Process:
1. **Mark Phase**: Mark all reachable objects
2. **Sweep Phase**: Free unmarked objects
3. **Compact Phase** (optional): Compact memory to reduce fragmentation

Mark Phase:

```
void mark_phase(GenerationalGC* gc) {
    // 1. Mark roots
    for (root in gc->roots) {
        mark_object(root);
    }

    // 2. Mark transitively
    while (mark_stack not empty) {
        object = pop(mark_stack);
        if (!object->marked) {
            object->marked = true;
            for (field in object->fields) {
                if (is_pointer(field)) {
                    push(mark_stack, field);
                }
            }
        }
    }
}
```

Sweep Phase:

```
void sweep_phase(GenerationalGC* gc) {
    object = old_gen->start;

    while (object < old_gen->end) {
        if (object->marked) {
            // Keep object, clear mark
            object->marked = false;
            object = next_object(object);
        } else {
            // Free object
            size = object->size;
            add_to_free_list(object, size);
            object = skip_object(object, size);
        }
    }
}
```

Performance:
- Pause time: 10-100 ms
- Throughput: 90-95% (10-5% GC overhead)
- Frequency: Every 100-1000 MB allocated

Full Collection

When: Triggered when old generation is full or on explicit request.

Process:
1. Minor GC (collect young generation)
2. Major GC (collect old generation)
3. Compact (optional, if fragmentation high)

Write Barriers

Write barriers track references from old generation to young generation.

Purpose

Without write barriers, we would need to scan the entire old generation during minor GC. Write barriers allow us to track only the relevant references.

Implementation

Card Marking:

```
void write_barrier(void* old_obj, void* new_value) {
    if (is_old_generation(old_obj) && is_young_generation(new_value)) {
        // Mark card
        size_t card_index = ((uintptr_t)old_obj - old_gen_start) / CARD_SIZE;
        card_table[card_index] = DIRTY;
    }
}
```

Card Table:
- Divide old generation into cards (typically 512 bytes)
- One byte per card (clean/dirty)
- Scan only dirty cards during minor GC

Overhead:
- Memory: ~0.2% (1 byte per 512 bytes)
- Time: 1-2 instructions per write

Root Set Management

Root Types

Stack Roots:
- Local variables
- Function parameters
- Temporary values

Global Roots:
- Global variables
- Static variables
- Constants

JIT Roots:
- References in compiled code

- Inline caches
- Code metadata

Root Scanning

```
typedef struct {
    GCObject** roots;
    size_t count;
    size_t capacity;
} GCRootSet;

void gc_add_root(GenerationalGC* gc, GCObject** root) {
    if (gc->roots->count >= gc->roots->capacity) {
        resize_root_set(gc->roots);
    }
    gc->roots->roots[gc->roots->count++] = root;
}
```

Memory Allocation

Young Generation Allocation

Fast Path (bump pointer):

```
void* gc_alloc_young(GenerationalGC* gc, size_t size) {
    void* result = gc->young_gen->eden_top;
    void* new_top = result + size;

    if (new_top <= gc->young_gen->eden_end) {
        gc->young_gen->eden_top = new_top;
        return result;
    }

    // Slow path: trigger minor GC
    gc_collect_young(gc);
    return gc_alloc_young(gc, size);
}
```

Performance: 2-5 CPU cycles (fast path)

Old Generation Allocation

Free List Allocation:

```
void* gc_alloc_old(GenerationalGC* gc, size_t size) {
    // Find free block
    FreeBlock* block = find_free_block(gc->old_gen->free_list, size);

    if (block != NULL) {
        // Allocate from free block
        void* result = block->start;

        if (block->size > size + MIN_BLOCK_SIZE) {
            // Split block
            split_free_block(block, size);
        } else {
            // Use entire block
            remove_from_free_list(block);
        }

        return result;
    }

    // No suitable block: trigger major GC
    gc_collect_full(gc);
    return gc_alloc_old(gc, size);
}
```

Performance: 10-50 CPU cycles

Performance Characteristics

Allocation Performance

Generation	Allocation Time	Throughput
Young	2-5 cycles	~1 GB/s
Old	10-50 cycles	~100 MB/s

Collection Performance

Collection Type	Pause Time	Frequency	Throughput
Minor GC	1-10 ms	Every 1-10 MB	95-99%
Major GC	10-100 ms	Every 100-1000 MB	90-95%
Full GC	20-200 ms	Rare	85-90%

Memory Overhead

Component	Overhead
Object Header	24 bytes per object
Card Table	0.2% of old generation
Free List	1-5% of old generation
GC Metadata	1-2% of total heap
Total	**10-20%**

Tuning Parameters

Heap Sizing

```
// Create GC with custom sizes
GenerationalGC* gc = generational_gc_create(
    256 * 1024,   // Young generation: 256 KB
    768 * 1024    // Old generation: 768 KB
);
```

Guidelines:
- Young generation: 10-30% of total heap
- Old generation: 70-90% of total heap
- Larger young generation: Fewer minor GCs, longer pauses
- Smaller young generation: More minor GCs, shorter pauses

Collection Thresholds

```
// Set GC threshold (percentage of nursery)
gc->gc_threshold = 80;  // Trigger at 80% full
```

Trade-offs:
- Higher threshold: Better throughput, longer pauses
- Lower threshold: Shorter pauses, lower throughput

Promotion Threshold

```
// Set promotion age threshold
gc->promotion_threshold = 3;  // Promote after 3 survivals
```

Trade-offs:
- Higher threshold: Fewer promotions, more minor GC work
- Lower threshold: More promotions, larger old generation

Debugging and Profiling

GC Statistics

```
void enhanced_vm_get_gc_stats(
    const EnhancedVM* vm,
    uint64_t* gc_collections,
    uint64_t* gc_bytes_allocated,
    uint64_t* gc_bytes_freed,
    size_t* young_used,
    size_t* old_used
);
```

Tracked Metrics:
- Total collections (minor + major)
- Bytes allocated
- Bytes freed
- Current heap usage
- Collection pause times

GC Logging

```
// Enable GC logging
gc_enable_logging(gc, true);

// Log output:
// [GC] Minor collection: 2.3 ms, 1.2 MB collected
// [GC] Major collection: 45.7 ms, 15.3 MB collected
```

Memory Safety

Safety Guarantees

1. **No Dangling Pointers**: GC ensures objects are not freed while referenced
2. **No Memory Leaks**: Unreachable objects are automatically freed
3. **No Use-After-Free**: Objects remain valid while reachable
4. **No Double-Free**: GC manages all deallocation

Limitations

1. **Finalizers**: Not supported (use explicit cleanup)
2. **Weak References**: Not supported (future enhancement)
3. **Manual Memory Management**: Not available (by design)

Future Enhancements

Planned Features

1. **Concurrent GC**: Reduce pause times with concurrent collection
2. **Incremental GC**: Spread collection work over time
3. **Weak References**: Support for caches and observers
4. **Finalizers**: Cleanup hooks for resources

5. **Compaction**: Reduce fragmentation in old generation

Research Directions

1. **Region-Based GC**: Allocate objects in regions
2. **Reference Counting**: Hybrid RC + tracing GC
3. **Escape Analysis**: Stack-allocate non-escaping objects

References

- System Design (system-design.md)
- VM Architecture (vm-architecture.md)
- JIT Architecture (jit-architecture.md)
- API Documentation (../api/html/index.html)

BDI Kernel Team
October 2024